

Homework 8: Structured sparsity in genetics

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyana Slavchev
Görkem Kadir Solun

Problem 19: Exponential family of the negative binomial distribution

Part a: Proof that the negative binomial distribution belongs to the exponential family and definition of all its components.

A distribution belongs to the exponential family if its density can be written as:

$$p(y|\theta) = \exp(\eta(\theta)\tau(y) + h(y) - \alpha(\theta))$$

or in its canonical form as:

$$p(y|\eta) = \exp(\eta\tau(y) + h(y) - \alpha(\eta))$$

The negative binomial distribution can be rewritten as:

$$\begin{aligned}
f(x|p) &= \binom{x+r-1}{x} p^x (1-p)^r \\
&= \frac{(x+r-1)!}{x!(x+r-1-x)!} p^x (1-p)^r \\
&= \frac{(x+r-1)!}{x!(r-1)!} p^x (1-p)^r \\
&= \exp \left[\ln \left(\frac{(x+r-1)!}{x!(r-1)!} p^x (1-p)^r \right) \right] \\
&= \exp [\ln((x+r-1)!) - \ln(x!) - \ln((r-1)!) + x \ln(p) + r \ln(1-p)] \\
&= \exp \left[\underbrace{x}_{\tau(x)} \underbrace{\ln(p)}_{\eta(p)} + \underbrace{\ln((x+r-1)!) - \ln(x!)}_{h(x)} - \underbrace{(\ln((r-1)!) - r \ln(1-p))}_{\alpha(p)} \right] \\
\Rightarrow p(x|p) &= \exp [\eta(p)\tau(x) + h(x) - \alpha(p)]
\end{aligned}$$

To summarise:

$$\begin{aligned}
\eta(p) &= \ln(p) \\
\tau(x) &= x \\
h(x) &= \ln((x+r-1)!) - \ln(x!) \\
\alpha(p) &= \ln((r-1)!) - r \ln(1-p)
\end{aligned}$$

Alternatively, the term $\ln((r-1)!)$, which does not depend on p or x , and is therefore constant, could also be included in $h(x)$ as $-\ln((r-1)!)$

Part b: Expected value manipulation and derivation

Now, we derive the expected value of a Negative Binomial random variable. We first apply the definition of expected value for a discrete probability distribution:

$$\begin{aligned}
\sum_{x=1}^{\infty} x f(x|p) &= \sum_{x=1}^{\infty} \binom{x+r-1}{x} p^x (1-p)^r x \\
&= \sum_{x=1}^{\infty} \frac{(x+r-1)!}{x!(r-1)!} p^x (1-p)^r x \\
&= \frac{(1-p)^r}{(r-1)!} \sum_{x=1}^{\infty} \frac{(x+r-1)!}{(x-1)!} p^x
\end{aligned}$$

Substituting $y = x - 1$ yields:

$$\begin{aligned}
&= \frac{(1-p)^r}{(r-1)!} \sum_{y=0}^{\infty} \frac{(y+r)!}{y!} p^{y+1} \\
&= \frac{rp}{(1-p)} \sum_{y=0}^{\infty} \frac{(y+r)!}{y!r!} p^y (1-p)^{r+1} \\
&= \frac{rp}{(1-p)} \underbrace{\sum_{y=0}^{\infty} \binom{y+(r+1)-1}{y} p^y (1-p)^{r+1}}_{=1}
\end{aligned}$$

Notice that the term inside of the brackets corresponds to the density function of a Negative Binomial distribution $NB(r+1, p)$. We are summing this over all supported values of the distribution, which equals to 1 according to the definition of a density function.

From this, we conclude that the expected value of a Negative Binomial random variable is:

$$\mathbb{E}[X] = \frac{rp}{(1-p)}$$

Rewriting the exponential family formulation in terms of the expected value:

$$\mathbb{E}[X] = \frac{rp}{(1-p)} = \mu \implies rp = \mu - \mu p \implies p = \frac{\mu}{r + \mu}$$

For this definition of p, also $1 - p$ can be derived:

$$1 - p = 1 - \frac{\mu}{r + \mu} = \frac{r + \mu - \mu}{r + \mu} = \frac{r}{r + \mu}$$

Substituting these results into the exponential form:

$$f(x|p) = \exp \left[\underbrace{\tilde{x}}_{\tau(x)} \underbrace{\ln(p)}_{\eta(p)} + \underbrace{\ln((x+r-1)!) - \ln(x!)}_{h(x)} - \underbrace{(\ln((r-1)!) - r \ln(1-p))}_{\alpha(p)} \right]$$

$$\Rightarrow f(x|\mu) = \exp \left[\underbrace{\tilde{x}}_{\tau(x)} \underbrace{\ln\left(\frac{\mu}{r+\mu}\right)}_{\eta(\mu)} + \underbrace{\ln((x+r-1)!) - \ln(x!)}_{h(x)} - \underbrace{\left(\ln((r-1)!) - r \ln\left(\frac{r}{r+\mu}\right)\right)}_{\alpha(\mu)} \right]$$

Note that the natural parameter η is not the identity link of μ for the Negative Binomial: the notation “ $\eta := \mu$ ” in the exercise is understood as a reparameterization in terms of $\mu = \mathbb{E}[X]$, yielding $\eta(\mu) = \ln\left(\frac{\mu}{r+\mu}\right)$.

Problem 20: Sparsity and metabolic engineering in *B. subtilis*

Part a: Data preparation

```
suppressPackageStartupMessages(library("glmnet"))
```

Warning: package 'glmnet' was built under R version 4.5.3

```
# loading the data
riboflavin <- read.csv("riboflavin.csv")
```

```
set.seed(42)
```

```
# splitting into training and test set
```

```
samples <- sample(c(TRUE,FALSE), nrow(riboflavin), replace = TRUE, prob = c(0.7,0.3))
```

```
r_train <- riboflavin[samples,]
r_test <- riboflavin[!samples,]
```

```
x_train <- as.matrix(subset(r_train, select = -y))
y_train <- as.numeric(r_train$y) # cv.glmnet expects a list, not a matrix, so no subset func
```

```
x_test <- as.matrix(subset(r_test, select = -y))
y_test <- as.numeric(r_test$y)
```

Part b: Model fitting

Part c: Sparsity and performance

Part b and c are completed together below.

```
# creating a matrix that will hold alphas and all results
model_alphas <- c(1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.5, 0.0, 0.0, 0.0)
res <- array(model_alphas, dim = c(4,3))
colnames(res) <- c("Lasso regression", "Ridge regression", "Elastic net")
rownames(res) <- c("alpha", "optimal lambda", "number of non-zero coefficients", "MSE on test set")

set.seed(42)

for (model in colnames(res)) {
  cv_model <- cv.glmnet(x_train, y_train, alpha = res[1, model], nfolds = 10)
  print(cv_model)

  # part b, optimal lambda
  cat("Optimal lambda for", model, " is", cv_model$lambda.min, ".\n")
  res[2, model] <- cv_model$lambda.min

  plot(cv_model)

  # part c, non zero coefficient (expect intercept)
  res[3, model] <- cv_model$nzzero[cv_model$lambda == cv_model$lambda.min]

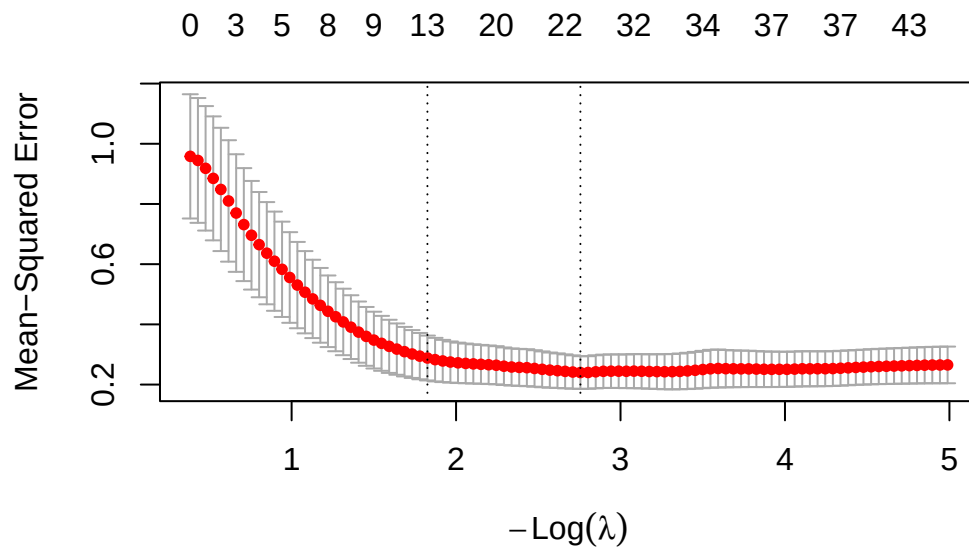
  # part c, MSE of prediction on test set
  test_pred <- predict(cv_model, newx = x_test, s = "lambda.min")
  res[4, model] <- mean((test_pred - y_test)^2)
}
```

Call: cv.glmnet(x = x_train, y = y_train, nfolds = 10, alpha = res[1, model])

Measure: Mean-Squared Error

	Lambda	Index	Measure	SE	Nonzero
min	0.06353	52	0.2400	0.0543	26
1se	0.16107	32	0.2883	0.0745	13

Optimal lambda for Lasso regression is 0.06353033 .

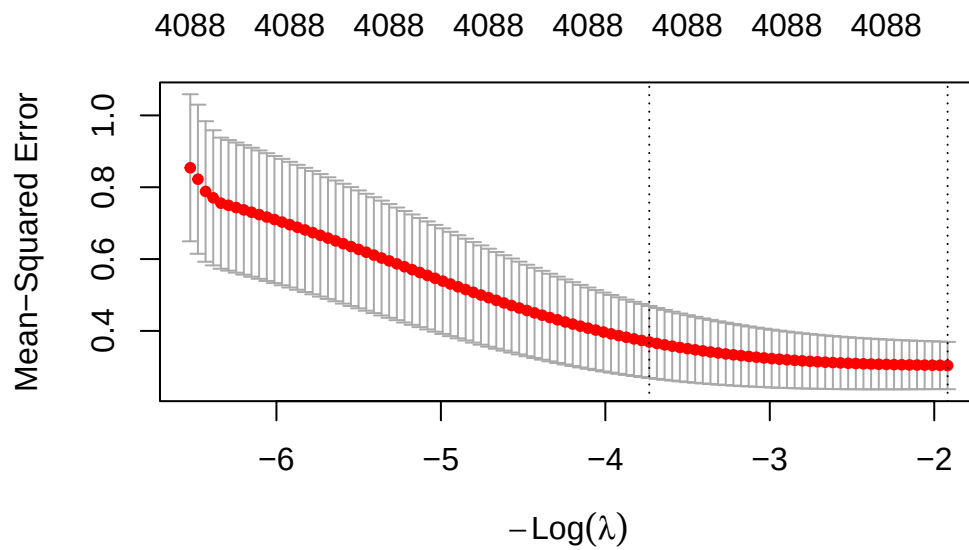


```
Call: cv.glmnet(x = x_train, y = y_train, nfolds = 10, alpha = res[1,      model])
```

Measure: Mean-Squared Error

	Lambda	Index	Measure	SE	Nonzero
min	6.81	100	0.3036	0.06586	4088
1se	41.80	61	0.3691	0.10026	4088

Optimal lambda for Ridge regression is 6.812149 .

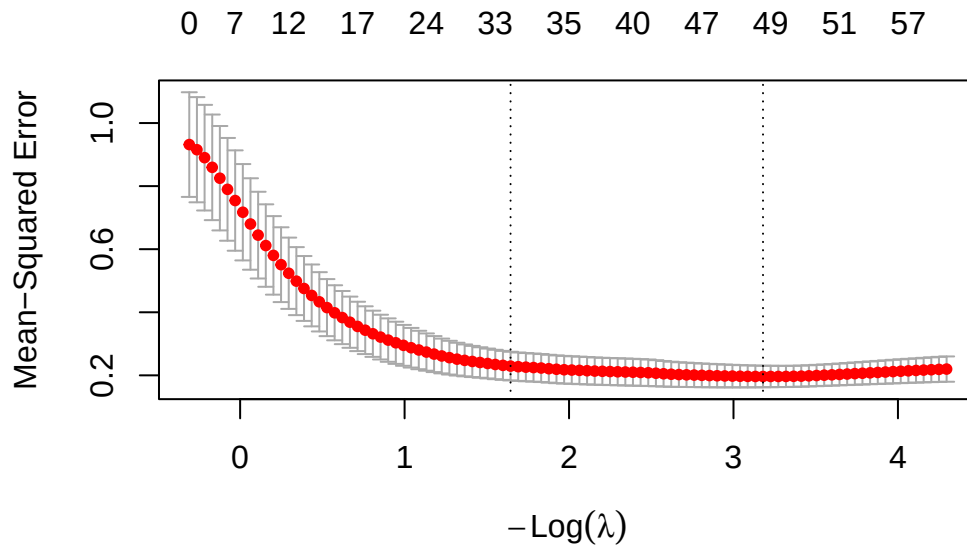


Call: `cv.glmnet(x = x_train, y = y_train, nfolds = 10, alpha = res[1, model])`

Measure: Mean-Squared Error

	Lambda	Index	Measure	SE	Nonzero
min	0.04161	76	0.1963	0.03417	48
1se	0.19312	43	0.2293	0.04546	34

Optimal lambda for Elastic net is 0.04160664 .



```
print(res)
```

	Lasso regression	Ridge regression	Elastic net
alpha	1.00000000	0.00000000	0.50000000
optimal lambda	0.06353033	6.8121487	0.04160664
number of non-zero coefficients	26.00000000	4088.00000000	48.00000000
MSE on test set	0.32956910	0.2471255	0.29397729

Ridge regression obtains the lowest mean squared error (MSE), 0.247, of the three models. However, as per definition, it does not drive any of the coefficients to zero. Infact, it has 4088 non zero coefficients, while in comparison lasso regression has only 26, which results in a worse MSE of 0.330. Elastic net, which is a combination of the models, is sensibly in between: it results in 48 non zero coefficients, more than lasso, and many fewer than ridge, and an MSE of 0.294, again in between the three.

Problem 21: Identification of protein-protein interaction networks via the graphical lasso

Part a: Raw data loading

```
#loading the data
pe_wX <- read.csv("proteinexpression.csv")
pe <- subset(pe_wX, select = -X)
protein_names <- colnames(pe)
```

Part b: Lasso model fitting

```
set.seed(42)

# matrix to hold the coefficients: rows = (Intercept) + all proteins, columns = response protein
coefs <- matrix(0.0, nrow = length(protein_names) + 1, ncol = length(protein_names))
colnames(coefs) <- protein_names
rownames(coefs) <- c("(Intercept)", protein_names)

for (protein in protein_names) {
  prot_idx_bool <- protein == protein_names
  x_train <- as.matrix(pe[, !prot_idx_bool]) # all other proteins as predictors
  y_train <- as.numeric(pe[, prot_idx_bool]) # response: the selected protein

  cv_model <- cv.glmnet(x_train, y_train, alpha = 1, nfolds = 5)
  coef_set <- coef(cv_model, s = "lambda.1se")

  # place each coefficient in the row matching its predictor name;
  # the diagonal entry (protein as its own predictor) stays 0 by construction
  dense_coefs <- as.matrix(coef_set)
  coefs[rownames(dense_coefs), protein] <- dense_coefs[, 1]
}
```

```
coefs
```

	Raf	Erk	Plcg	PKC
(Intercept)	1.854444e-16	3.293702e-16	-4.053521e-16	-2.145421e-16
Raf	0.000000e+00	0.000000e+00	0.000000e+00	-1.106461e-01
Erk	0.000000e+00	0.000000e+00	2.068917e-02	2.085326e-01
Plcg	3.019400e-02	2.031032e-02	0.000000e+00	-1.264114e-02
PKC	-5.201830e-02	2.042725e-01	0.000000e+00	0.000000e+00
PKA	-1.044486e-01	1.282242e-01	-2.747001e-01	-7.314515e-06
PIP2	0.000000e+00	0.000000e+00	3.952955e-01	9.468949e-03
PIP3	0.000000e+00	-9.119959e-03	0.000000e+00	0.000000e+00

Mek	7.617789e-01	-4.360909e-01	0.000000e+00	1.159712e-01
P38	0.000000e+00	-3.516740e-02	4.014542e-02	5.450508e-01
Jnk	-1.165059e-01	6.679169e-02	6.595628e-02	2.541578e-01
Akt	0.000000e+00	7.706429e-01	1.403766e-01	-1.315520e-01
	PKA	PIP2	PIP3	Mek
(Intercept)	-3.684898e-16	4.173081e-16	-1.469868e-16	-7.026392e-17
Raf	-1.654303e-01	0.000000e+00	0.000000e+00	5.814105e-01
Erk	1.371713e-01	0.000000e+00	-1.967937e-02	-2.617117e-01
Plcg	-3.455345e-01	5.197890e-01	0.000000e+00	0.000000e+00
PKC	-8.378922e-03	0.000000e+00	0.000000e+00	8.343856e-02
PKA	0.000000e+00	-2.721087e-02	1.220152e-01	0.000000e+00
PIP2	-3.130417e-02	0.000000e+00	3.776500e-01	0.000000e+00
PIP3	6.795809e-02	2.669876e-01	0.000000e+00	0.000000e+00
Mek	-4.437583e-02	0.000000e+00	0.000000e+00	0.000000e+00
P38	-2.441638e-01	0.000000e+00	0.000000e+00	0.000000e+00
Jnk	-4.687199e-02	0.000000e+00	-1.354832e-01	2.029117e-01
Akt	0.000000e+00	0.000000e+00	-6.614323e-02	2.794392e-01
	P38	Jnk	Akt	
(Intercept)	8.613119e-17	1.218670e-16	-2.226731e-16	
Raf	6.394925e-02	-1.615249e-01	0.000000e+00	
Erk	0.000000e+00	8.869667e-02	6.314930e-01	
Plcg	3.442772e-02	9.847771e-02	8.598334e-02	
PKC	4.508902e-01	3.050015e-01	-7.120003e-02	
PKA	-1.632394e-01	-3.176402e-02	-2.846906e-02	
PIP2	0.000000e+00	0.000000e+00	0.000000e+00	
PIP3	0.000000e+00	-7.141943e-02	0.000000e+00	
Mek	1.296387e-02	3.070484e-01	4.171852e-01	
P38	0.000000e+00	2.137944e-01	9.643674e-02	
Jnk	1.490349e-01	0.000000e+00	0.000000e+00	
Akt	8.611193e-02	0.000000e+00	0.000000e+00	

Part c: Building the interaction matrix

```
# creating the interaction matrix im
im <- matrix(0, nrow = length(protein_names), ncol = length(protein_names))
colnames(im) <- protein_names
rownames(im) <- protein_names

# filling the interaction matrix (both the upper and lower triangle), the same protein is not
for (protein in protein_names) {
  for (protein_2 in protein_names) {
```

```

    if (protein != protein_2) {
      if ((coefs[protein,protein_2] != 0) & (coefs[protein_2,protein] != 0)) {
        im[protein,protein_2] <- 1
        im[protein_2,protein] <- 1
      }
    }
  }
}

```

```
im
```

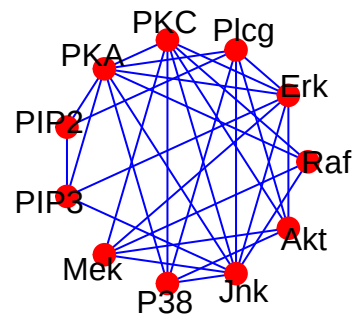
	Raf	Erk	Plcg	PKC	PKA	PIP2	PIP3	Mek	P38	Jnk	Akt
Raf	0	0	0	1	1	0	0	1	0	1	0
Erk	0	0	1	1	1	0	1	1	0	1	1
Plcg	0	1	0	0	1	1	0	0	1	1	1
PKC	1	1	0	0	1	0	0	1	1	1	1
PKA	1	1	1	1	0	1	1	0	1	1	0
PIP2	0	0	1	0	1	0	1	0	0	0	0
PIP3	0	1	0	0	1	1	0	0	0	1	0
Mek	1	1	0	1	0	0	0	0	0	1	1
P38	0	0	1	1	1	0	0	0	0	1	1
Jnk	1	1	1	1	1	0	1	1	1	0	0
Akt	0	1	1	1	0	0	0	1	1	0	0

Part d: Network plotting

```

source("plotcirculargraph.R")
plot_circular_graph(im)

```



From the interaction graph it appears that PIP2 is the protein that interacts with the least number of other proteins, 3, while PKA and Jnk are the protein that interact with the most: 8.